TLS and VPN Security Configuration Analyzer

Project Synopsis — Cryptography and Network Security

Submitted by: Siddharth Gupta(590015808) | Lakshya Ratudi(590018503)

Guide: *Parnika Bhatt*

Department: SOCS-CSF

Semester: *V*

1. Project Title

TLS and VPN Security Configuration Analyzer: Detecting Cryptographic Weaknesses and Measuring the Effect of Remediation

2. Abstract

TLS and VPNs protect most modern network communication, yet their security depends as much on *configuration* as on the strength of the underlying algorithms. AES is secure, but AES in a deprecated protocol version, with a 1024-bit RSA key, or with certificate validation disabled, is not. This project builds a Python-based analyzer that reads TLS server settings (from a configuration file, a sample JSON profile, or a live handshake against a test server) and OpenVPN/WireGuard configuration files. It extracts the cryptographic parameters in use, compares them against a JSON rule base derived from public guidance (NIST SP 800-52r2, NIST SP 800-57, RFC 8446, RFC 8996, Mozilla's server-side TLS guidelines), assigns severity levels, explains each weakness in plain language, suggests a corrected configuration, re-evaluates it, and reports a 0–100 score before and after remediation. The goal is to demonstrate, measurably, how cryptographic configuration choices affect real-world security. The project analyses configurations only; it does not exploit systems.

3. Introduction

Secure communication aims to provide four properties over an untrusted network: *confidentiality* (only the intended parties read the data), *integrity* (tampering is detected), *authentication* (each party knows who it is talking to), and, ideally, *forward secrecy* (past sessions stay safe even if long-term keys leak).

TLS (Transport Layer Security) is the protocol behind HTTPS and many other services. In a handshake, the client and server (1) agree on a protocol version and *cipher suite*, (2) the server proves its identity using an X.509 *certificate* signed by a trusted Certificate Authority (CA), (3) both sides run a *key exchange* (e.g., ECDHE) to derive shared *session keys*, and (4) bulk data is protected with symmetric encryption and integrity protection (usually an AEAD cipher such as AES-GCM or ChaCha20-Poly1305). A cipher suite is a named bundle of these choices, e.g., `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384` = ECDHE key exchange, RSA authentication, AES-256-GCM encryption, SHA-384 for the key-derivation hash.

A VPN (Virtual Private Network) creates an encrypted tunnel between two endpoints so traffic crosses the Internet privately. **OpenVPN** uses TLS for the control channel and a configurable data-channel cipher and HMAC; **WireGuard** uses a fixed modern suite (Curve25519, ChaCha20-Poly1305, BLAKE2s) and has far fewer settings to get wrong.

Why configuration matters. Cryptographic algorithms are only as strong as the way they are used. Real examples: POODLE exploited SSL 3.0 fallback; BEAST/Lucky13 targeted CBC modes in old TLS; Sweet32 attacked 64-bit block ciphers (3DES, Blowfish) over long sessions; FREAK/Logjam exploited export-grade and small Diffie-Hellman parameters; collision attacks made MD5/SHA-1 certificate signatures forgeable in principle. In every case the weak option was usually *enabled by configuration* when better options existed. An administrator who leaves legacy settings on for "compatibility" can silently nullify the strength of the modern algorithms enabled next to them, because an attacker can try to negotiate down to the weakest option the server accepts.

4. Problem Statement

Organizations and students configure TLS and VPN services using long, option-heavy files (Nginx/Apache `ssl_*` directives, OpenSSL cipher strings, OpenVPN `.ovpn` files). Common problems include:

- **Deprecated protocol versions** (SSL 3.0, TLS 1.0, TLS 1.1) still enabled.
- **Weak cipher suites** (NULL, EXPORT, RC4, DES/3DES, CBC-mode suites without need).
- **Weak encryption or insecure hashing** (single DES, Blowfish-64, MD5, SHA-1 for signatures/HMAC).
- **Poor certificate configuration**: expired, self-signed in production, hostname mismatch, SHA-1/MD5 signature, RSA key below 2048 bits, overly long validity.
- **Insecure key exchange**: static RSA key transport (no forward secrecy), small or non-standard DH groups (<2048 bits).

- **Missing certificate validation** on the VPN or client side (`remote-cert-tls` / `verify-x509-name` absent, `verify=false` in code).
- **Weak VPN settings**: `cipher BF-CBC` , `auth MD5/SHA1` , no `tls-crypt` / `tls-auth` , `comp-lzo` (compression-related risks), `tls-version-min` unset.

Manual review is hard because the rules span multiple standards, cipher strings must be expanded into real suites (`HIGH:!aNULL` hides what is actually negotiated), settings interact, and guidance changes over time. Manual checks are slow, inconsistent and rarely produce a measurable before/after comparison. An automated analyzer gives repeatable, explainable results and doubles as a learning tool.

5. Motivation

- Misconfiguration, not broken mathematics, is a very common cause of cryptographic failure in deployed systems.
- Existing tools (testssl.sh, SSL Labs, Nessus) are powerful but opaque for learning; students rarely see *why* a rule exists or how to build one.
- Building the analyzer forces us to understand cipher-suite structure, certificates, key exchange and VPN parameters in depth, which is exactly the course content.
- A before/after score makes the benefit of correct cryptographic choices concrete and demonstrable in a viva.

6. Objectives

1. Parse TLS and VPN (OpenVPN, WireGuard) configurations and extract security-relevant parameters.
2. Detect deprecated, weak or misconfigured cryptographic settings using an updatable JSON rule base.
3. Classify each finding as Critical, High, Medium, Low or Secure and explain the cryptographic reason.
4. Recommend and generate a corrected configuration.
5. Re-evaluate the corrected configuration and compute a documented 0–100 score before and after.
6. Present results through a simple dashboard and downloadable report.

7. Proposed System

The analyzer accepts a configuration (or a lab TLS endpoint), converts it into a normalized *profile* of cryptographic parameters, runs the profile through a rule engine, computes risk and score, generates recommendations plus a corrected profile, re-runs the engine on the corrected profile, and reports the difference.

Pipeline steps:

1. **Input**: upload/paste a config file, choose a bundled sample, or enter `host:port` of a test server.
2. **Parse/collect**: read directives, or perform a handshake with Python's `ssl` module and fetch the certificate.
3. **Extract**: build a normalized profile (versions, suites, key exchange, key sizes, hashes, certificate fields, VPN parameters).
4. **Compare** with the rule base (JSON).
5. **Detect** violations and assign severity (Critical, High, Medium, Low, or Secure).
6. **Explain** each finding (each rule includes its cryptographic rationale).
7. **Recommend** a stronger setting for each finding.
8. **Re-evaluate** the corrected profile.
9. **Compare** scores and findings before vs. after.
10. **Report** as HTML/PDF/JSON.

8. Scope of the Project

In scope

- TLS: protocol versions, cipher suites, key exchange, key sizes, certificate fields (expiry, signature algorithm, key type/size, SAN/hostname, self-signed, validity period) from config files, sample profiles and a **local/lab TLS server**.
- VPN: **OpenVPN** `.ovpn` /server `.conf` files (primary) and **WireGuard** `.conf` files (small secondary checks). OpenVPN is chosen because it has many configurable cryptographic settings, so there is more to analyze; WireGuard is included mainly to show a "secure by design" contrast.
- Rule base, scoring, recommendations, before/after comparison, report and dashboard.

Out of scope

- Exploitation, packet attacks, brute forcing, downgrade attack execution, or any penetration testing of third-party systems.
- Implementing cryptographic primitives ourselves (we use standard libraries).
- IPsec/IKEv2, SSH, SMTP STARTTLS (future scope).
- Vulnerabilities in software implementations (e.g., Heartbleed-class bugs), firewall or OS hardening.
- Guaranteeing overall system security.

Configuration analysis vs. exploitation. We *read* settings and compare them with published guidance. We never attempt to break the encryption or attack a service. The only network activity is an ordinary TLS handshake against servers we own or are permitted to test (e.g., a local OpenSSL `s_server`).

9. Major Features

9.1 Core / MVP features

#	Feature	How it can be implemented
1	TLS version detection	Parse `ssl_protocols` (Nginx) / `SSLProtocol` (Apache) / sample JSON; for a live server, attempt handshakes with `ssl.SSLContext` pinned to each version and record accepted ones.
2	Cipher-suite analysis	Expand OpenSSL cipher strings via `openssl ciphers -v 'STRING'` (subprocess) or a bundled suite table; split each suite into KX / auth / cipher / MAC fields.
3	Encryption algorithm analysis	Look up the cipher field (AES-GCM, AES-CBC, 3DES, RC4, ChaCha20) in the rule base.
4	Hash/MAC analysis	Check MAC/PRF hash (MD5, SHA-1, SHA-256/384) and certificate signature hash.
5	Key-exchange analysis	Classify as ECDHE/DHE (forward secret), static RSA (no FS), or anonymous; check DH group size when given.

#	Feature	How it can be implemented
6	Key-size / strength check	Map key sizes to approximate security strength using NIST SP 800-57 (e.g., RSA-2048 ≈ 112 bits, RSA-3072 ≈ 128, P-256 ≈ 128).
7	Certificate analysis	Load PEM with the `cryptography` library: public key type/size, signature algorithm, validity dates, SAN, issuer = subject (self-signed).
8	Expiry check	Compare `not_valid_after` to today; warn when under 30 days left.
9	Certificate signature algorithm check	Read `signature_hash_algorithm`; flag MD5/SHA-1.
10	OpenVPN configuration analysis	Parse directives (`cipher`, `data-ciphers`, `auth`, `tls-version-min`, `tls-cipher`, `tls-crypt`, `remote-cert-tls`, `comp-lzo`, `dh`).
11	Deprecated algorithm detection	Rule base has a `deprecated` flag per algorithm.
12	Severity classification	Each rule carries a severity.
13	Human-readable explanations	Each rule has a `reason` and `fix` text.
14	Recommended secure settings	Rule `fix` fields feed a remediation generator that rewrites the profile.
15	Before/after comparison	Re-run the engine on the remediated profile; diff findings and scores.
16	Security score	Weighted deduction model (Section 16).
17	Report generation	Jinja2 HTML template, export to JSON and print-to-PDF.
18	Simple dashboard	Streamlit app (upload, results table, score display, before/after view).

9.2 Optional advanced features (only if time permits)

- Live TLS scanning of a *local* test server including the negotiated cipher and presented certificate chain.

- WireGuard checks (PresharedKey usage, `AllowedIPs` review, key presence).
- Certificate chain validation against the system trust store and hostname verification.
- Export a corrected `nginx.conf` / `.ovpn` file ready to use.
- Mapping findings to references (NIST guideline section, CWE-327 "Use of a Broken or Risky Cryptographic Algorithm", CWE-326 "Inadequate Encryption Strength").
- Scan history stored in SQLite with trend charts.

10. Cryptographic Concepts Used

Concept	Brief explanation	Where it appears in the project
Symmetric encryption	One shared key encrypts and decrypts bulk data; fast.	Cipher-field analysis of every suite and VPN `cipher`.
AES	128-bit block cipher, 128/192/256-bit keys; secure in GCM or CTR/CBC+HMAC modes.	Marked secure in GCM; CBC rated lower; key length checked.
DES / 3DES	DES has a 56-bit key (broken); 3DES has a 64-bit block, vulnerable to Sweet32.	Rule base entries used as legacy/insecure comparison cases; flagged High/Critical.
Asymmetric cryptography	Key pairs (public/private) for authentication and key establishment.	Certificate public keys, handshake authentication.
RSA	Based on integer factorization; needs ≥2048-bit keys today (3072 for long-term).	Key-size check on certificates; static-RSA key exchange flagged.
ECC	Elliptic-curve keys give similar strength with shorter keys (P-256, X25519).	Curve and key-size checks; ECDSA certificates rated secure.
Diffie-Hellman / ECDHE	Two parties derive a shared secret over an open channel; "E" means ephemeral keys.	Key-exchange classification, DH group size check, forward-secrecy rule.
Hash functions	One-way fixed-size digests (collision/preimage resistance).	Certificate signature hash, HMAC/PRF checks; MD5 and SHA-1 flagged.

Concept	Brief explanation	Where it appears in the project
SHA family	SHA-1 (deprecated), SHA-2 (256/384/512), SHA-3.	Rule base marks SHA-1 weak, SHA-256+ acceptable.
MAC / HMAC / AEAD	Tag proving integrity and authenticity under a shared key; AEAD combines encryption + MAC.	VPN `auth` setting; AEAD vs. CBC+HMAC distinction in TLS suites.
Digital signatures	Private-key signature verified with public key (RSA-PSS, ECDSA).	Certificate signature algorithm check.
Digital certificates	X.509 binds an identity to a public key, signed by a CA.	Certificate parser: subject, SAN, validity, issuer.
PKI	CAs, chains of trust, revocation.	Self-signed detection, chain/hostname checks (optional chain validation).
Session keys	Short-lived symmetric keys derived per connection.	Explained in handshake model; ephemeral key-exchange rule.
Forward secrecy	Compromise of long-term keys does not reveal past sessions.	ECDHE/DHE required; static RSA penalized.
TLS handshake	Negotiation of version, suite, authentication, keys.	Live handshake collector and sample profile structure.
VPN encryption	Tunnel with control-channel TLS and data-channel cipher/HMAC.	OpenVPN parser, WireGuard fixed-suite contrast.

11. System Architecture

Flow: User/Input → Configuration Parser / TLS Collector → Parameter Extraction (normalized profile) → Rule Engine → Risk Analyzer → Recommendation Engine → Score Calculator → Report/Dashboard

Components

1. **Input layer**: file upload, pasted text, sample selector, or `host:port` of a test server.

2. **Parsers/collectors**: `tls_parser` (Nginx/Apache/JSON), `openvpn_parser`, `wireguard_parser`, `tls_scanner` (handshake via `ssl`), `cert_parser` (`cryptography`).
3. **Parameter extraction**: converts everything into one normalized dictionary (versions, suites, key exchange, auth, cipher, MAC, key sizes, cert fields, VPN fields) so rules do not depend on the input format.
4. **Rule engine**: loads `rules.json`, evaluates each rule against the profile, emits findings.
5. **Risk analyzer**: groups findings by category, applies severity and score caps (e.g., any Critical finding limits the maximum score).
6. **Recommendation engine**: maps findings to fixes; builds a corrected profile by applying them.
7. **Score calculator**: computes per-category and overall scores for the original and corrected profiles.
8. **Report/dashboard**: presents score, findings, explanations, remediation and before/after comparison.

Flowchart (Mermaid; paste into any Mermaid editor to render):

```
flowchart TD
    A[User Input: config file / sample / host:port] --> B{Input type?}
    B -->|TLS config| C[TLS Config Parser]
    B -->|OpenVPN / WireGuard| D[VPN Config Parser]
    B -->|Test server| E[TLS Handshake Collector]
    C --> F[Normalized Crypto Profile]
    D --> F
    E --> F
    F --> G[Rule Engine - rules.json]
    G --> H[Risk Analyzer - severity and categories]
    H --> I[Score Calculator - before score]
    H --> J[Recommendation Engine]
    J --> K[Corrected Profile]
    K --> G2[Rule Engine - re-evaluation]
    G2 --> L[Score Calculator - after score]
    I --> M[Before vs After Comparison]
    L --> M
    M --> N[Report and Dashboard]
```

12. Methodology / Working Procedure

Phase	Activities	Output
1. Research & requirements	Study TLS 1.2/1.3, OpenVPN options, NIST SP 800-52r2 / 800-57, Mozilla TLS guidelines, RFC 8996.	Requirement list
2. Rule selection	Choose 40–60 rules across TLS and VPN; assign severity and rationale; cite a source for each.	`rules.json` v1
3. Sample dataset	Prepare 10–15 sample configs: weak, medium, secure TLS (Nginx/Apache + JSON), OpenVPN weak/secure, WireGuard; generate test certificates with OpenSSL (expired, SHA-1, 1024-bit, self-signed, strong).	`samples/` folder
4. TLS analysis module	Config parser, cipher-string expander, certificate analyzer, optional handshake collector for a local server.	`tls_module.py`
5. VPN parser	Directive parser for OpenVPN; INI parser for WireGuard.	`vpn_module.py`
6. Rule engine	Generic evaluator for JSON conditions.	`engine.py`
7. Risk scoring	Weighted deduction model and caps.	`scoring.py`
8. Recommendation engine	Rule-to-fix mapping; corrected profile generator.	`remediate.py`
9. UI/dashboard	Streamlit pages: input, results, before/after.	`app.py`
10. Testing	Run all test cases; fix false positives/negatives; unit tests with `pytest`.	Test report
11. Before/after evaluation	Remediate every weak sample and tabulate score changes.	Evaluation table
12. Documentation & demo	Report, README, slides, demo script.	Final submission

Demonstration plan: (a) analyze a weak Nginx config and a weak `.ovpn` file, (b) show findings and explanations, (c) apply remediation, (d) show the score rising, (e) optionally scan a local `openssl s_server` with and without legacy versions enabled.

13. Technology Stack

Technology	Why it is needed
Python 3.11+	Easy to read, rich security libraries, works well on macOS.
`ssl` module (standard library)	Perform real TLS handshakes to a lab server; read negotiated version/cipher.
`cryptography` library	Parse X.509 certificates safely (key size, signature algorithm, dates) without hand-written ASN.1 parsing.
OpenSSL (CLI)	Generate test keys/certs, run a local test TLS server (`openssl s_server`), expand cipher strings (`openssl ciphers -v`). macOS ships LibreSSL, so install OpenSSL 3 with Homebrew.
Config parsing (regex + `configparser`)	Parse Nginx/Apache directives, OpenVPN lines, WireGuard INI. Simple enough that a full grammar parser is unnecessary.
JSON	Stores the rule base in an editable, readable form.
SQLite (optional)	Scan history for the optional trend feature.
Streamlit	Fastest way to build a Python dashboard (file upload, tables, charts) without frontend code. Flask + Bootstrap is an alternative if a custom UI is wanted.
Jinja2	HTML report templates.
pytest	Automated test cases for rules and scoring.
Git/GitHub	Version control, collaboration, evidence of progress for evaluation.

Technology	Why it is needed
OpenVPN / WireGuard installation	*Not required.* We analyze config files only, so no VPN server needs to be running.

14. Security Rule Engine (Design)

Each rule is a JSON object with an identifier, the field it inspects, a condition, a severity, an explanation and a fix. The engine loads the file and evaluates every rule against the normalized profile.

```
[
  {"id":"TLS-001","category":"protocol","field":"tls.versions",
   "condition":{"contains_any":["SSLv3","TLSv1.0","TLSv1.1"]},
   "severity":"High",
   "reason":"TLS 1.0/1.1 rely on MD5/SHA-1 based constructions and weak CBC
handling and were formally deprecated by RFC 8996.",
   "fix":"Allow only TLSv1.2 and TLSv1.3."},

  {"id":"CIPH-002","category":"cipher","field":"tls.suites[].cipher",
   "condition":{"in":["3DES","DES","RC4","NULL"]},
   "severity":"High",
   "reason":"64-bit block ciphers are vulnerable to birthday attacks
(Sweet32); DES has a 56-bit key; RC4 has statistical biases.",
   "fix":"Use AES-128-GCM, AES-256-GCM or ChaCha20-Poly1305."},

  {"id":"CERT-003","category":"certificate","field":"cert.days_remaining",
   "condition":{"lt":0},"severity":"High",
   "reason":"An expired certificate cannot be trusted; clients may reject it
or users may learn to ignore warnings.",
   "fix":"Renew and deploy a new certificate."},

  {"id":"CERT-004","category":"certificate","field":"cert.rsa_bits",
   "condition":{"lt":2048},"severity":"High",
   "reason":"RSA below 2048 bits provides less than about 112 bits of
security (NIST SP 800-57).",
   "fix":"Generate a new RSA-3072 or ECDSA P-256 key."},

  {"id":"KX-005","category":"key_exchange","field":"tls.suites[].kx",
   "condition":{"equals":"RSA"},"severity":"Medium",
   "reason":"Static RSA key transport has no forward secrecy.",
```

```
   "fix":"Prefer ECDHE (or DHE with at least 2048-bit groups)."},

  {"id":"VPN-006","category":"vpn","field":"openvpn.cipher",
   "condition":{"in":["BF-CBC","DES-CBC","DES-EDE3-CBC"]},"severity":"High",
   "reason":"64-bit block ciphers in the data channel are exposed to Sweet32-
style attacks.",
   "fix":"Use AES-256-GCM (data-ciphers AES-256-GCM:AES-128-GCM:CHACHA20-
POLY1305)."}
]
```

Positive ("secure") rule. A composite rule marks the configuration as *Secure* only when *all* of the following hold: versions ⊆ {TLS 1.2, TLS 1.3}, every offered suite uses an AEAD cipher, every key exchange is ECDHE/DHE (or TLS 1.3), the certificate key is RSA ⩾ 2048 or ECDSA ⩾ P-256, the signature hash is SHA-256 or stronger, and the certificate is valid.

Engine logic (pseudocode):

```
load rules.json
for each rule:
    value = get(profile, rule.field)   # may be a list
    if evaluate(rule.condition, value): add Finding(rule.id, severity,
reason, fix)
if no Critical/High/Medium findings and secure_rule(profile): status = Secure
```

Because rules are data, a faculty member or future user can add or change a rule without editing Python code, and every rule cites its source for traceability.

15. Sample Demonstration Scenarios

Scenario 1: Deliberately weak TLS configuration

Input (Nginx):

```
ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
ssl_ciphers  DES-CBC3-SHA:AES128-SHA:RC4-SHA;
ssl_certificate server.crt;   # RSA-1024, SHA-1 signature, expired
```

Detected weakness	Cryptographic reason	Severity	Fix
TLS 1.0 and 1.1 enabled	Deprecated; weak PRF and CBC issues; downgrade exposure	High	`ssl_protocols TLSv1.2 TLSv1.3;`
`DES-CBC3-SHA` (3DES)	64-bit block, Sweet32	High	Remove; use AES-GCM suites
`RC4-SHA`	Keystream biases; prohibited by RFC 7465	Critical	Remove
Static RSA key exchange in all suites	No forward secrecy; one key leak exposes all recorded traffic	Medium	Use ECDHE suites
RSA-1024 certificate key	About 80-bit security, below NIST minimum	High	Reissue with RSA-3072 or ECDSA P-256
SHA-1 signature	Collision attacks make forgery feasible in principle	High	Reissue with SHA-256
Expired certificate	Authenticity cannot be verified	High	Renew

Corrected configuration:

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-
ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-
POLY1305:ECDHE-RSA-CHACHA20-POLY1305;
ssl_prefer_server_ciphers off;
ssl_certificate server.crt;   # RSA-3072 or ECDSA P-256, SHA-256, valid
```

Improvement (illustrative): about **26/100 (capped at 40, Critical Risk)** → about **96/100 (Secure)**.

Scenario 2: Secure TLS configuration

Input: TLS 1.2 + 1.3 only, ECDHE-AEAD suites only, ECDSA P-256 certificate, SHA-256 signature, 90-day validity.

Check	Result
Protocol	Secure
Cipher	AEAD only, secure
Key exchange	ECDHE, forward secrecy
Certificate	Valid, strong key, SHA-256
Findings	None (at most informational notes such as HSTS or OCSP stapling)

Result: 95–100/100 — Secure. No remediation is required. This scenario shows the analyzer does not raise false alarms on a good configuration.

Scenario 3: OpenVPN configuration with weak settings

Input (`client.ovpn`):

```
cipher BF-CBC
auth MD5
comp-lzo
dh dh1024.pem
# no remote-cert-tls, no tls-crypt, no tls-version-min
```

Detected weakness	Cryptographic reason	Severity	Fix
`cipher BF-CBC`	64-bit block, Sweet32	High	`data-ciphers AES-256-GCM:AES-128-GCM:CHACHA20-POLY1305`
`auth MD5`	Obsolete hash, weak legacy practice	High	`auth SHA256`
`dh dh1024.pem`	1024-bit DH is within reach of large-scale precomputation (Logjam)	High	2048+ bit DH, or `ecdh-curve prime256v1`
No server-certificate verification	Active man-in-the-middle possible	High	`remote-cert-tls server` and `verify-x509-name`

Detected weakness	Cryptographic reason	Severity	Fix
No `tls-crypt`	Control channel exposed to probing and some DoS	Medium	`tls-crypt ta.key`
`tls-version-min` missing	Might negotiate below TLS 1.2	Medium	`tls-version-min 1.2`
`comp-lzo` enabled	Compression before encryption can leak plaintext information (VORACLE)	Medium	Remove compression

Improvement (illustrative): about **30/100 (Critical Risk)** → about **92/100 (Secure)**. A WireGuard sample is included as a contrast: fixed modern primitives mean very few findings, mostly operational ones (review of `AllowedIPs`, use of a PresharedKey).

Note: scores in the scenarios are illustrative and will be replaced by the tool's actual output during evaluation.

16. Security Scoring Model

This is a **project-defined, transparent model**, not an industry standard. It borrows the general idea of weighted categories used by public scanners, but the weights and caps are our own design decisions, documented and configurable.

Category weights (TLS profile)

Category	Weight
Protocol versions	25
Cipher / encryption strength	25
Key exchange and forward secrecy	15
Certificate security (key size, validity, trust)	20
Hash / signature security	15
Total	**100**

VPN profile uses VPN-specific groups: data-channel cipher (30), authentication/HMAC (15), TLS control channel and version (20), key exchange/DH (15), certificate verification and `tls-crypt` (20).

Method

1. Each category starts at its full weight.
2. Each finding deducts a fraction of that category's weight by severity: Critical = 100%, High = 60%, Medium = 30%, Low = 10%.
3. A category cannot go below 0.
4. **Caps:** any Critical finding caps the total at 40; any High finding caps it at 70, reflecting that one serious flaw can defeat the rest of the design.
5. Rating bands: 90–100 **Secure**, 75–89 **Low Risk**, 50–74 **Medium Risk**, 25–49 **High Risk**, 0–24 **Critical Risk**.

Worked example (Scenario 1). Protocol: High (−15) → 10/25. Cipher: Critical (RC4) → 0/25. Key exchange: Medium (−4.5) → 10.5/15. Certificate: expired + weak key (two High findings, floored at 0) → 0/20. Hash: High (−9) → 6/15. Raw total about 26.5, which is already below the Critical cap of 40, so the rating is **Critical Risk**. After remediation every category returns to full weight, apart from small informational deductions, giving about 96.

Defensibility. Weights are justified by attack impact: protocol and cipher failures can expose all traffic, certificate failures undermine authentication. They are stored in `scoring.json` so they can be tuned, and the report states that the score is a *relative indicator for comparing configurations*, not a certification.

17. Testing and Evaluation Strategy

Testing uses intentionally weak and secure configurations, checked with `pytest` and manually in the dashboard. Metrics: detection correctness (expected findings present), false positives on secure samples (target: zero), and score improvement after remediation.

ID	Test Case	Configuration	Expected Detection	Recommended Fix	Expected Result
T1	Deprecated protocols	`ssl_protocols TLSv1 TLSv1.1;`	TLS 1.0/1.1 flagged High	Use TLS 1.2/1.3	Finding removed; protocol score full
T2	SSLv3 enabled	SSLv3 in protocol list	Critical (obsolete)	Remove SSLv3	Score rises, cap lifted
T3	3DES suite	`DES-CBC3-SHA`	Weak cipher (Sweet32), High	AES-GCM	Cipher category recovers
T4	RC4 / NULL / EXPORT	`RC4-SHA`, `NULL-SHA`, `EXP-*`	Critical	Remove	Score no longer capped at 40
T5	No forward secrecy	RSA key-transport suites only	Medium	ECDHE suites	Key-exchange score full
T6	Expired certificate	Cert with past `notAfter`	High	Renew	Certificate finding cleared
T7	Weak RSA key	RSA-1024 cert	High	RSA-3072 / ECDSA P-256	Key-size finding cleared
T8	SHA-1 certificate	SHA-1 signed cert	High	Reissue with SHA-256	Hash score full
T9	Self-signed / hostname mismatch	Self-signed cert, wrong SAN	Medium/High	Trusted CA cert with correct SAN	Trust finding cleared
T10	Weak OpenVPN cipher/auth	`cipher BF-CBC`, `auth MD5`	High (two findings)	AES-256-GCM, SHA256	Findings cleared
T11	Missing cert verification	No `remote-cert-tls`	High	Add `remote-cert-tls server`	VPN auth score full
T12	Secure TLS baseline	TLS 1.2/1.3, ECDHE+AEAD, ECDSA	None	None	Score ≥ 90, Secure, zero false positives
T13	Secure OpenVPN baseline	AES-256-GCM, tls-crypt, tls-version-min 1.2	None	None	Score ≥ 90
T14	Malformed config	Empty or garbled file	Graceful error	Ask for valid input	No crash

Additional evaluation: remediate every weak sample, tabulate before/after scores, and compare the tool's findings with a reference scan (e.g., `nmap --script ssl-enum-ciphers` or testssl.sh) on the same *local* test server.

18. Expected Results (Final Product)

The user supplies a configuration or lab target and receives output such as:

```
Security Score: 48/100 — HIGH RISK

Detected issues:
 - Deprecated protocol version (TLS 1.0 enabled)
 - Weak cipher (3DES)
 - Weak key exchange (static RSA, no forward secrecy)
 - Certificate problem (1024-bit RSA key, SHA-1 signature)

Recommendations:
 - Disable deprecated protocols
 - Replace weak ciphers with AES-GCM / ChaCha20-Poly1305
 - Enable ECDHE key exchange
 - Reissue certificate with RSA-3072 or ECDSA P-256 and SHA-256

After applying secure settings:
Security Score: 90+/100 — SECURE
```

The central message of the demonstration is that **the same strong algorithms become secure or insecure depending on configuration choices**, and that this difference can be measured.

19. Expected Outcomes

Academic

- Real-world understanding of symmetric/asymmetric cryptography, hashing, MACs, signatures and key exchange.
- Understanding of the TLS handshake, certificates and PKI.
- Understanding of VPN encryption and how OpenVPN uses TLS.
- Knowledge of how algorithms and parameters are selected and retired in real systems (NIST, RFCs).

Practical

- A working tool that identifies weak cryptographic configurations.
- Practice with secure configuration and certificate management.
- Measurable before/after evidence that remediation improves security.

- Experience in parsing, rule design, testing and report generation.

20. Advantages

- Automated, repeatable and explainable analysis.
- Rules are data in JSON, so they can be updated as standards change.
- Works offline on config files; no need to touch real systems.
- Teaches *why* each setting is risky, not just that it is.
- Lightweight stack that runs on a student laptop (macOS).
- Before/after scoring makes improvements visible.

21. Limitations

- Static analysis of configuration only; it cannot detect software vulnerabilities, compromised keys, weak randomness or operational mistakes.
- A good score does not guarantee overall system security; it applies only to the cryptographic configuration reviewed.
- Rules reflect guidance at a point in time and must be maintained.
- Parsers cover common directives of Nginx/Apache/OpenVPN/WireGuard, not every vendor syntax or include chain.
- A configured option does not always equal negotiated behavior (client capabilities, proxies, load balancers). Live scanning is limited to what Python's `ssl` module and the local OpenSSL build support; some legacy protocols may be disabled locally, so detecting them relies on config analysis or a separately built OpenSSL.
- The score is a project-defined indicator, not an industry standard.
- Live scanning is restricted to lab servers we own; no scanning of third-party systems.

22. Future Scope

These are not part of the current implementation.

- Live scanning of public domains with permission and rate control.
- Automated VPN auditing across many endpoints; IPsec/IKEv2 and SSH support.
- CVE/CWE integration, linking findings to known attacks and weakness classes.
- Auto-generated, ready-to-deploy corrected configuration files.

- Natural-language explanations produced by a language model *only if* it clearly improves clarity, with the rule base remaining the source of truth.
- Organization-wide scanning with inventory and dashboards.
- Compliance mapping (NIST, PCI DSS, ISO 27001 controls).
- Continuous monitoring with alerts on configuration drift.
- Integration with SIEM or security dashboards.

23. Conclusion

The TLS and VPN Security Configuration Analyzer turns abstract cryptography concepts into a practical, measurable tool. By checking protocol versions, cipher suites, key exchange, certificates and VPN parameters against a transparent rule base, and then showing that remediation improves the score, the project demonstrates that secure communication depends on correct configuration as much as on strong algorithms. The scope is deliberately limited to analysis rather than exploitation, making it realistic for undergraduate implementation and ethical to demonstrate.

24. References

1. E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF, 2018.
2. T. Dierks and E. Rescorla, *The TLS Protocol Version 1.2*, RFC 5246, IETF, 2008.
3. K. Moriarty and S. Farrell, *Deprecating TLS 1.0 and TLS 1.1*, RFC 8996, IETF, 2021.
4. A. Popov, *Prohibiting RC4 Cipher Suites*, RFC 7465, IETF, 2015.
5. NIST SP 800-52 Rev. 2, *Guidelines for the Selection, Configuration, and Use of TLS Implementations*, 2019.
6. NIST SP 800-57 Part 1 Rev. 5, *Recommendation for Key Management*, 2020.
7. NIST SP 800-131A Rev. 2, *Transitioning the Use of Cryptographic Algorithms and Key Lengths*, 2019.
8. Mozilla, *Server Side TLS* guidelines and SSL Configuration Generator.
9. OpenVPN Community, *OpenVPN Reference Manual*.
10. J. Donenfeld, *WireGuard: Next Generation Kernel Network Tunnel*, NDSS 2017.
11. K. Bhargavan and G. Leurent, *On the Practical (In-)Security of 64-bit Block Ciphers (Sweet32)*, ACM CCS 2016.

12. D. Adrian et al., *Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice (Logjam)*, ACM CCS 2015.
13. Python documentation: `ssl` module; `cryptography` library documentation (cryptography.io).
14. MITRE, CWE-326 and CWE-327 (cwe.mitre.org).

Verify exact titles, years and details when finalizing the list in your university's citation format.

C. Development Roadmap (about 10–12 weeks)

Week	Milestone	Deliverable
1	Study TLS, certificates, OpenVPN options; read NIST SP 800-52r2 and Mozilla guidelines; set up Git repo, Python and OpenSSL on macOS	Notes, repo skeleton
2	Finalize 40–60 rules with severity and rationale; define the profile data model	`rules.json` v1, profile schema
3	Build sample dataset: weak/secure Nginx, OpenVPN, WireGuard configs; generate test certificates with OpenSSL	`samples/` folder
4	TLS config parser and cipher-suite expander	`tls_module.py` + unit tests
5	Certificate analyzer using `cryptography`	`cert_module.py` + tests
6	OpenVPN and WireGuard parsers	`vpn_module.py` + tests
7	Rule engine and severity classification	`engine.py`; CLI findings output
8	Scoring model and recommendation/remediation engine	`scoring.py`, `remediate.py`; first before/after run
9	Streamlit dashboard and HTML/JSON report	Working UI (MVP complete)
10	Full test suite (T1–T14), bug fixes, optional local-server live scan	Test report
11	Evaluation table; optional extras (WireGuard checks, config export)	Evaluation results
12	Documentation, slides, demo rehearsal	Final report and demo

Tip: get a thin end-to-end version working by Week 7 (one weak config → findings → score), then widen coverage. This protects you if time runs short.